



Build

Task	Command
Full build	./gradlew build
Clean build	./gradlew clean build
Run checks	./gradlew check
Skip tests	./gradlew build -x test
Checkstyle main	./gradlew checkstyleMain
Checkstyle test	./gradlew checkStyleTest

Test

Task	Command
Run all tests	./gradlew test
Run a specific test class	./gradlew --tests CasePartyLinkControllerTest
Run a specific test class	./gradlew --tests "MyTest.whenThisThenThat"
View tests ran	./gradlew test --info
See stack traces	./gradlew test --stacktrace
	./gradlew test --debug

Dependencies

Task	Command
Show dependency tree	./gradlew dependencies
Dependency information	./gradlew dependencyInsight --dependency spring-web



Task	Command
Print current directory	pwd
List files	ls
Detailed listing	-l ls -l
Include hidden files	-a ls -la
List files human readable	-h ls -lah
Change directory	cd cd Documents
Go up one directory	cd ..
Go to home directory	cd ~
Go to previous directory	cd -
Clear terminal	clear
Show history	history
Open manual	man man ls
Find executable	which which java
Current user	whoami
Current date/time	date

Task	Command
Create empty file	touch touch file.txt
Create directory	mkdir mkdir MyDirectory
Create nested directory	mkdir -p mkdir -p MyDirectory/img
Create file containing text	echo > echo "Hello" > file.txt
Copy file	cp cp source.txt destination.txt
Copy file into directory	Cp cp source.txt ~/Documents
Copy directory	cp -r cp -r sourceDir DestinationDir
Rename	mv mv old.txt new.txt
Move file	mv mv file.txt ~/Documents
Rename and Move	mv mv old.txt ~/Documents/new.txt
Delete file	rm rm file.txt
Delete empty directory	rmdir rmdir MyDirectory
Delete a directory and its contents	rm -r rm MyDirectory
Force deletion	rm -f rm -f file.txt



Task	Command
Force and Recursive delete	rm -rf rm -rf MyDirectory
Print entire file	cat cat file.txt
View large file	less less file.txt
Page down (less)	Space
Page up (less)	b
Search (less)	/text
Next Search result (less)	n
Quit (less)	q
View first 20 lines	head -20 head -20 file.txt
Last 20 lines	tail -20 tail -20 file.txt
Show end of log and continually update	tail -f tail -f application.log

Grep

Task	Command
Search file	grep grep "ERROR" application.log
Case insensitive	-i grep -i "error" application.log

Task	Command
Search recursively	-r grep -r "TODO" .
Current directory	. grep -r "TODO" .
Show line numbers	-n grep -n "ERROR" application.log
Exclude directory	--exclude-dir= grep -r "TODO" . --exclude-dir="MyDirectory"
Search specific file types	--include= grep -r "TODO" . --include="*.java"
Find a filename	find -name find . -name "application.yml"
File all Java files	find -name find . -name "*.java"
Search case insensitive	-iname find . -iname "*.java"
Find directories	find . -type d
Find files	find . -type f
Find files modified recently	find . -type f -mtime -<days> find . -type -mtime -1
Find large files	find . -type f -size + find . -type f -size +100M
Populate file	echo > echo "hello" > file.txt
Append to file contents	echo >> echo "another line" >> file.txt

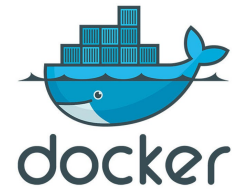


Task	Command
Use file as input	< command < file.txt
Redirect errors	2> command 2> errors.txt
Redirect standard output and errors	&> command &> output.txt
Take output of one command and give to another (ls → grep)	 ls -la grep ".java"
	docker ps grep wiremock
	cat application.log grep ERROR head
Count TODO matches	grep -r "TODO" wc -l
Count lines in file	wc -l wc -l file.txt
Count words in file	wc -w wc -w file.txt
Count characters in file	wc -c wc -c file.txt
Sort	sort sort file.txt
Reverse sort	-r sort -r file.txt
Numerical sort	-n sort -n numbers.txt

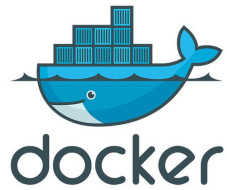
Task	Command
Remove duplicates	uniq sort file.txt uniq
Count duplicates	uniq -c sort file.txt uniq -c
View variable	echo echo \$PATH
Set variable (current shell)	MY_VARIABLE= MY_VARIABLE="Hello"
Export variable for child processes	export export MY_VARIABLE="Hello"
List processes	ps
More comprehensive	ps aux
Search processes	ps aux grep java
Kill process	kill kill <PID>
Force kill	kill -9 kill -9 <PID>
Filesystem usage	df -h
Directory size	du -sh .
Size of each item	du -sh *
Create script	touch touch script.sh
Run script	./ ./script.sh
Make HTTP request	curl curl https://google.com



Task	Command
Test connectivity	ping <code>ping www.google.com</code>
Check port	nc -vz <code>nc -vz localhost 8080</code>
Find what's using a port	lsof -i <code>lsof -i :8080</code>
Kill whatever is using port	kill \$(lsof -t -i) <code>kill \$(lsof -t -i :8080)</code>
Show TCP connections	netstat -an -p tcp
Show listening ports	netstat -an grep LISTEN
Check specific port	netstat -an grep netstat -an grep './8080'
See all network connections	netstat -an



Task	Command
List running containers	docker ps
List all containers	docker ps -a
Start a container	docker start <code>docker start <container></code>
Stop a container	docker stop <code>docker stop <container></code>
Restart a container	docker restart <code>docker restart <container></code>
Remove a container	docker rm <code>docker rm <container></code>
Force remove a container	docker rm -f <code>docker rm -f <container></code>
See container details	docker inspect <code>docker inspect <container></code>
Check container's processes	docker top <code>docker top <container></code>
Check resource usage	docker stats
Check resource usage on one container	docker stats <code>docker stats <container></code>
Container logs	docker logs <code>docker logs <container></code>
Follow logs	docker logs -f <code>docker logs -f <container></code>



Task	Command
Remove stopped containers	<code>docker container prune</code>
Remove unused images	<code>docker image prune</code>
Remove unused networks	<code>docker network prune</code>
Remove unused volumes	<code>docker volume prune</code>
General cleanup	<code>docker system prune</code>
Aggressive cleanup	<code>docker system prune -a</code>

Image

Blueprint used to create containers. “Here’s everything needed to run a container.”

Container

The actual running instance of the image. Temporary.

Volume

The data associated with the container.

Think of it like a restaurant. Image = recipe, container = running copy of the image/recipe (food), volume = cupboard.
Containers are ephemeral, data/volume is persistent.



Task	Annotation
Register class as Spring bean	@Component
@Component for service/business	@Service
@Component for data-access code	@Repository
Spring MVC Controller	@Controller
Controller where methods return response bodies	@RestController
Class containing Spring configuration	@Configuration
Create Method Spring Bean	@Bean
Inject Dependency	@Autowired